Adani University, Ahmedabad

Faculty of Engineering Sciences and Technology (FEST)
Department of Computer Science and Engineering (AI-ML)

Project-Based Learning Proposal (Stage 1)

1. Course Details

Course Code	ECSCT24301
Course Name	Foundation of AI (B.Tech CSE AI/ML, Semester 5)
Academic Year	2026–27
Course Coordinator	Dr. Krishna H. Hingrajiya

#	Team Member Name	USN	Email ID
1	Jenil Mistry	2410111064	JENILMISTRY.CSE24@adaniuni.ac.in
2	Neel Parmar	2410111082	NEELKUMARPARMAR.CSE24@adaniuni.ac.in
3	Shreyansh Patel	2410111123	SHREYANSHPATEL.CSE24@adaniuni.ac.in

2. Project Title and Track

Title: *Parallel Alpha-Beta Search for an Othello Engine: A Comparative Study of Root Splitting, Principal Variation Splitting, and the Young Brothers Wait Concept*
Track: Applied AI Project Track

3. Problem Statement and Motivation

Othello (Reversi) is a two-player, zero-sum, deterministic, perfect-information game played on an 8×8 board. It has roughly 10^{28} legal positions, a game-tree complexity of about 10^{58}, and an average branching factor of about 10 over a game of at most 60 moves (Allis, 1994). Exhaustive search is therefore impossible, and strong play depends on depth-limited adversarial search with pruning. Alpha-beta pruning reduces the search cost from $O(b^d)$ to as little as $O(b^{d/2})$ when moves are well ordered (Knuth & Moore, 1975), but it is inherently sequential: the bounds (α, β) learned from earlier sibling subtrees are what cut off later ones. Searching siblings simultaneously therefore discards some of this information and causes *search overhead*, meaning that a parallel search visits nodes that a sequential search would have pruned (Marsland & Campbell, 1982; Brockington, 1996).

Formal statement. Let s_0 be an Othello position, d a fixed search depth, and P the number of parallel workers. Let $V_d(s_0)$ be the depth-d minimax value of s_0 under a fixed evaluation function e. The problem is to design and implement an engine that returns the same value and best move as the sequential alpha-beta search, while minimizing the wall-clock time $T_P(s_0, d)$:

$$\min_{\text{strategy}} \; T_P(s_0, d) \quad \text{subject to} \quad V_d^{(P)}(s_0) = V_d^{(1)}(s_0).$$

We study three parallelization strategies and answer three research questions. **RQ1:** how much of alpha-beta's pruning is lost under each strategy, measured as search overhead? **RQ2:** how do speedup and efficiency scale with P and depth d? **RQ3:** how does the quality of move ordering affect each strategy, and where does parallel scaling break down (failure analysis)?

4. Agent and Environment Characterization (PEAS)

Performance Measure	Win rate and mean final disc difference against baseline opponents; nodes generated/expanded; time per move; speedup S_P, efficiency E_P, and search overhead SO_P; agreement of the chosen move and value with the sequential search.
Environment	8×8 Othello board with two players. Two-player and competitive (adversarial), zero-sum, deterministic, fully observable, sequential, static, discrete, known rules. The search process runs on a shared-memory multicore machine with P worker threads.
Actuators	Placing a disc on a legal square (flipping all flanked discs) or passing when no legal move exists; outputting the chosen move to the GUI/protocol; internal scheduling of search tasks to workers.
Sensors	The full board (64 squares), the player to move, the move history, the remaining time budget, and the legal-move set produced by the move generator.

5. Formulation Details

5.1 Game formulation

- **State:** $s = (B, p)$, where $B \in \{\text{empty}, \text{black}, \text{white}\}^{8\times 8}$ and $p \in \{\text{black}, \text{white}\}$ is the player to move. States are stored as two 64-bit bitboards plus p.
- **Initial state:** the standard opening with four discs in the centre, black to move.
- **Actions $A(s)$:** the empty squares from which the current player flanks at least one opposing disc in one of 8 directions. If $A(s) = \emptyset$ the only action is PASS.
- **Transition model $\textsc{Result}(s, a)$:** place a disc on square a, flip all flanked opposing discs, and switch the player.
- **Terminal test:** the board is full or neither player has a legal move.
- **Utility:** at a terminal state, the final disc difference $U(s) = |\text{own}| - |\text{opp}|$ (win/loss/draw follows from its sign). Since this is a game rather than a path-finding task, the cost metric is this utility plus the computational cost (nodes, time, memory).

5.2 Evaluation function

For depth-limited search we use a linear evaluation $e(s) = w_1\,\Delta_{\text{mob}} + w_2\,\Delta_{\text{corner}} + w_3 \sum_i W_i x_i + w_4\,\Delta_{\text{stab}} + w_5\,\Delta_{\text{disc}}$, where Δ denotes the difference between the two players, x_i indicates occupancy of square i, and W is a positional weight matrix (corners high, squares next to corners negative). Weights depend on the game phase (Rosenbloom, 1982; Buro, 2002). Near the end of the game the evaluation equals the exact utility U, which gives ground truth for validating it.

5.3 Algorithmic terms and metrics

- **Minimax / negamax:** $V(s) = \max_a -V(\textsc{Result}(s, a))$; **alpha-beta:** the same value with a window (α, β) that prunes moves that cannot affect the result.
- **Node types** (Knuth & Moore, 1975): *PV* nodes lie on the principal variation, *CUT* nodes need only one child to cause a cutoff, and *ALL* nodes require every child to be searched. Parallel work is safe at ALL nodes and wasteful at CUT nodes.
- **Move ordering, iterative deepening, transposition table:** the better the ordering, the closer the search gets to $O(b^{d/2})$. The table uses Zobrist hashing (Zobrist, 1970); ordering uses the history heuristic (Schaeffer, 1989) and corner/mobility rules.
- **Metrics:** $S_P = T_1/T_P$ (speedup), $E_P = S_P/P$ (efficiency), $SO_P = N_P/N_1 - 1$ (search overhead, with N_P the total nodes visited), pruning efficiency $1 - N_{\alpha\beta}/N_{\text{minimax}}$, and effective branching factor $b^* = N^{1/d}$.

Figure 1 illustrates these ideas on a small tree: the first move (the eldest brother) is searched alone to establish α, and the remaining moves are then searched in parallel with that bound.

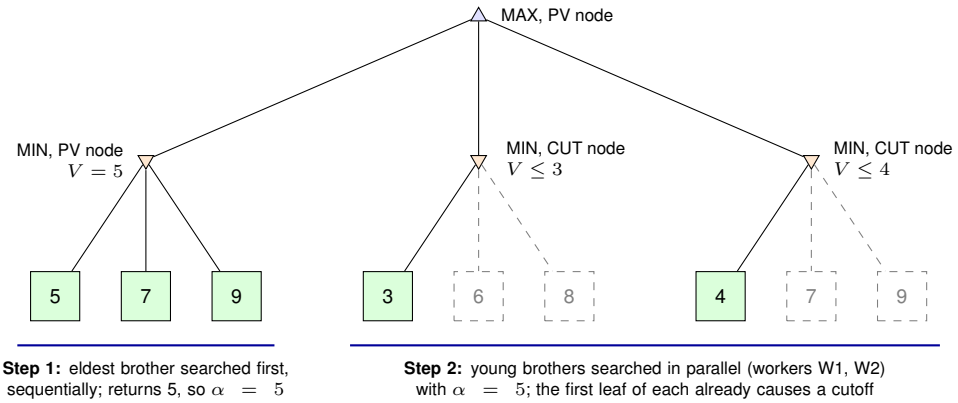


Figure 1: Depth-2 game tree under the Young Brothers Wait Concept. Dashed grey leaves are never searched because $\alpha = 5$ is already known. If B and C were started together with A (as in root splitting), they would not know α and would expand those leaves too, which is the search overhead described above.

6. Data and Literature Sourcing

Othello search needs test positions rather than a labelled dataset. We will use: (i) **FFO endgame test positions** from the French Othello Federation, which are standard benchmarks for solver speed and correctness (Fédération Française d'Othello, n.d.-a; [add verified URL]); (ii) **WTHOR** professional-game archives from the same federation, used to extract realistic opening and midgame positions and to compare our engine's choices with expert play (Fédération Française d'Othello, n.d.-b; [add verified URL]); (iii) **self-generated benchmark suites**: seeded random-play positions at fixed move numbers (early, mid, late game) so that every algorithm searches identical positions.

Literature base. Knuth & Moore (1975) analysed alpha-beta and its node types. Marsland & Campbell (1982) introduced principal variation splitting. Feldmann et al. (1993) proposed the Young Brothers Wait Concept, and Brockington (1996) gave a taxonomy of parallel game-tree search algorithms. Rosenbloom (1982) and Buro (2002) document strong Othello programs and evaluation functions, and Allis (1994) gives the complexity estimates. Russell & Norvig (2021) is the general reference for adversarial search. [Add 1–2 recent (2020 onward) papers on parallel or GPU-based game-tree search.]

7. Proposed Algorithmic Approaches

ID	Approach	Role in the study
B0	Random / greedy (max flips)	Naive baselines for playing strength.
B1	Plain depth-limited minimax	Baseline for node counts and pruning gain.
B2	Sequential alpha-beta + iterative deepening + transposition table + move ordering	Reference for correctness ($V_d^{(1)}$) and for T_1, N_1.
P1	Root splitting	Simplest parallel strategy: root moves are searched by different workers. Expected to show high search overhead.
P2	Principal Variation Splitting (PVS)	Search the leftmost (PV) path sequentially, then split the remaining siblings at each PV node among workers (Marsland & Campbell, 1982).
P3	Young Brothers Wait Concept (YBWC) with work stealing	At every node the eldest child is searched first; only then are the younger siblings searched in parallel, with idle workers stealing them (Feldmann et al., 1993).

Experimental plan. All engines use the same evaluation function so that differences come only from the search. We will run each parallel strategy with $P \in \{1, 2, 4, 8\}$ workers and depths from 6 to 12 on the benchmark suites, and record T_P, N_P, S_P, E_P, and SO_P. We will also run tournaments (with colour swaps) at equal time controls. For the failure analysis we will degrade move ordering on purpose and study cases where scaling flattens or overhead explodes. The correctness check is that P1–P3 return the same value as B2 at equal depth.

8. Planned Tooling and Libraries

- **Engine:** C++17 (CMake, g++/clang) with bitboard move generation, Zobrist hashing, and `std::thread` and atomics for parallel search. The move generator is validated with perft node counts.
- **Analysis and plotting:** Python 3 with NumPy, pandas, and Matplotlib for benchmark tables and plots (speedup vs. P, nodes vs. depth, pruning efficiency).
- **GUI / demo:** Pygame (or Streamlit) front end that talks to the engine over a simple text protocol on stdin/stdout, showing the board, the search statistics, and step-by-step decisions.
- **Collaboration and documentation:** Git/GitHub for code, Overleaf (LaTeX) for reports, APA style for all citations.

References

Allis, L. V. (1994). *Searching for solutions in games and artificial intelligence* [Doctoral dissertation, University of Limburg].

Brockington, M. G. (1996). A taxonomy of parallel game-tree search algorithms. *ICCA Journal, 19*(3), 162–174.

Buro, M. (2002). Improving heuristic mini-max search by supervised learning. *Artificial Intelligence, 134*(1–2), 85–99.

Feldmann, R., Mysliwietz, P., & Monien, B. (1993). Game tree search on a massively parallel system. In H. J. van den Herik, I. S. Herschberg, & J. W. H. M. Uiterwijk (Eds.), *Advances in computer chess 7*. University of Limburg. [verify page range]

Fédération Française d'Othello. (n.d.-a). *FFO endgame test suite* [Data set]. [URL and access date]

Fédération Française d'Othello. (n.d.-b). *WTHOR database of tournament games* [Data set]. [URL and access date]

Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence, 6*(4), 293–326.

Marsland, T. A., & Campbell, M. (1982). Parallel search of strongly ordered game trees. *ACM Computing Surveys, 14*(4), 533–551.

Rosenbloom, P. S. (1982). A world-championship-level Othello program. *Artificial Intelligence, 19*(3), 279–320.

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Schaeffer, J. (1989). The history heuristic and alpha-beta search enhancements in practice. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 11*(11), 1203–1212.

Zobrist, A. L. (1970). *A new hashing method with application for game playing* (Technical Report No. 88). University of Wisconsin, Computer Sciences Department.